

# HAPPYSTAYZ

Smart Hotel Management System

Project Documentation Report

Guest Self Check-In · Online Booking · Staff Attendance · Automated Payroll (Autopay) · Admin Dashboard

---

|                       |                                                         |
|-----------------------|---------------------------------------------------------|
| <b>Report Date:</b>   | July 14, 2026                                           |
| <b>Application:</b>   | HappyStayz Hotel Management Web App                     |
| <b>Stack:</b>         | Python 3.9+ · Flask 3.0 · SQLite · Jinja2 · Bootstrap 5 |
| <b>Seed Dataset:</b>  | Kaggle-style synthetic "Hotel Booking Demand" records   |
| <b>Prepared with:</b> | Flask, Werkzeug, SQLite3 (Python standard library)      |

## Executive Summary

HappyStayz is a working hotel management web application covering the full guest-to-staff-to-admin lifecycle of a small hotel: guests can browse room types, make a booking, and self check-in with a booking code to receive a digital room key instantly; staff clock themselves in and out from a personal dashboard, with late/present/absent status computed automatically from clock time; and administrators get a single dashboard for occupancy, bookings, staff attendance, and payroll.

The system's most distinctive feature is **autopay**: a payroll engine that converts logged attendance hours directly into salary, disburses payment automatically for any staff member with autopay enabled, and re-runs itself in the background on the 1st of every month without an administrator having to trigger it. The application is built on Flask and SQLite so it can be installed and run locally with a single command and no external database server, and it ships pre-seeded with 40 rooms, 10 staff members, 60 sample bookings drawn from a Kaggle-style dataset, and 30 days of attendance history so every screen has realistic data from the first run.

This report documents the system's architecture, data model, core modules, seeded demo data, the autopay automation logic, and recommendations for hardening the project beyond its current demo form.

# 1. Project Objective

Small hotels often run check-in, attendance, and payroll as three disconnected manual processes: a front-desk register for bookings, a paper or spreadsheet muster for staff attendance, and a separate, often delayed, manual salary calculation at month end. HappyStayz was built to demonstrate how a single lightweight web application can unify all three, so that:

- Guests can book and check themselves in without waiting on front-desk staff.
- Staff attendance is captured at the moment it happens, not reconstructed later.
- Payroll is derived directly from attendance data rather than recalculated by hand, removing a recurring manual task and the errors that come with it.

# 2. System Architecture

## 2.1 Technology stack

| Layer           | Technology                | Notes                                            |
|-----------------|---------------------------|--------------------------------------------------|
| Web framework   | Flask 3.0+                | Single app.py with route-based controllers       |
| Templating      | Jinja2 + Bootstrap 5      | Server-rendered HTML pages per role              |
| Database        | SQLite (file-based)       | No external DB server; auto-created on first run |
| Auth            | Werkzeug password hashing | Separate session scopes for staff and admin      |
| Background jobs | Python daemon thread      | Hourly check for month-start autopay run         |

## 2.2 Database schema

The schema is deliberately small — six tables — but relational: bookings reference rooms, attendance and payroll both reference staff, giving payroll a traceable path back to the individual clock-ins that produced it.

| Table       | Purpose                                                  | Key relationships                 |
|-------------|----------------------------------------------------------|-----------------------------------|
| rooms       | Room number, type, nightly price, status                 | Referenced by bookings            |
| bookings    | Guest details, dates, ADR, status, digital key code      | room_id → rooms                   |
| staff       | Staff identity, role, salary, bank account, autopay flag | Referenced by attendance, payroll |
| attendance  | Daily check-in/out time, hours worked, status            | staff_id → staff                  |
| payroll     | Per-month gross/net pay, status, transaction ref         | staff_id → staff                  |
| admin_users | Admin login credentials                                  | Standalone                        |

# 3. Core Modules & Features

### 3.1 Guest module

- **Room booking** — guest picks a room type and dates; the app assigns an available room, computes nights and total amount, and issues a booking code (e.g. *HSZ100000*).
- **Self check-in** — guest enters their booking code; on success the booking is marked `checked_in`, the room is marked occupied, and a digital key code is generated on the spot.

### 3.2 Staff module

- **Login** — username/password against hashed credentials in the staff table.
- **Self-service clock-in / clock-out** — a staff member's dashboard lets them check in and out; check-in before 10:00 is marked “present”, after 10:00 is marked “late”, and hours worked are computed automatically at check-out.
- **Personal history** — last 14 days of attendance and last 6 months of payroll, visible to the staff member themselves.

### 3.3 Admin module

- **Dashboard** — room occupancy, total bookings, staff present today, and running revenue.
- **Bookings** — filterable list of all bookings by status (confirmed, checked-in, checked-out, canceled, no-show).
- **Staff & attendance** — who is checked in today, 30-day attendance history per staff member, and a per-staff autopay ON/OFF toggle.
- **Payroll / autopay** — pick a month and run payroll on demand, or let the background worker run it automatically on the 1st of the month.

## 4. Seeded Demo Data

On first run, `database.py` creates the schema and seeds it with a fixed random seed, so every fresh install produces identical demo accounts and figures — useful for repeatable demos and grading.

### 4.1 Room inventory

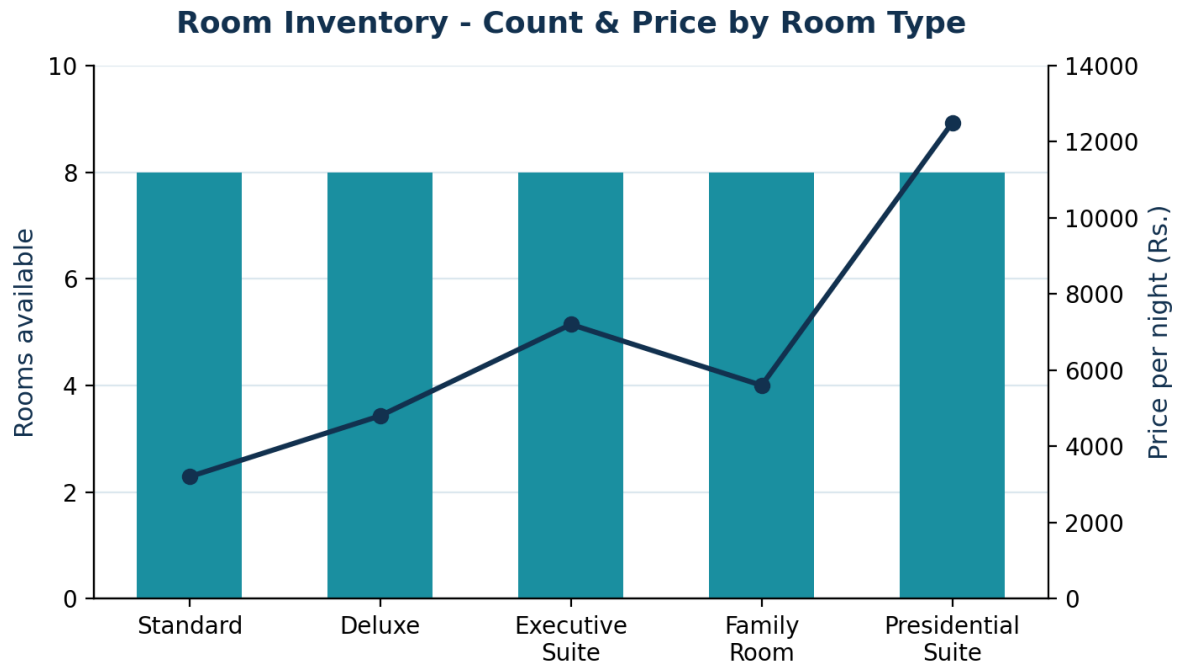


Figure 1. 40 rooms across 5 floors, 8 rooms per room type. Each floor is assigned a single room type, so price rises with floor level from Standard (Rs. 3,200) up to the Presidential Suite (Rs. 12,500).

### 4.2 Staffing

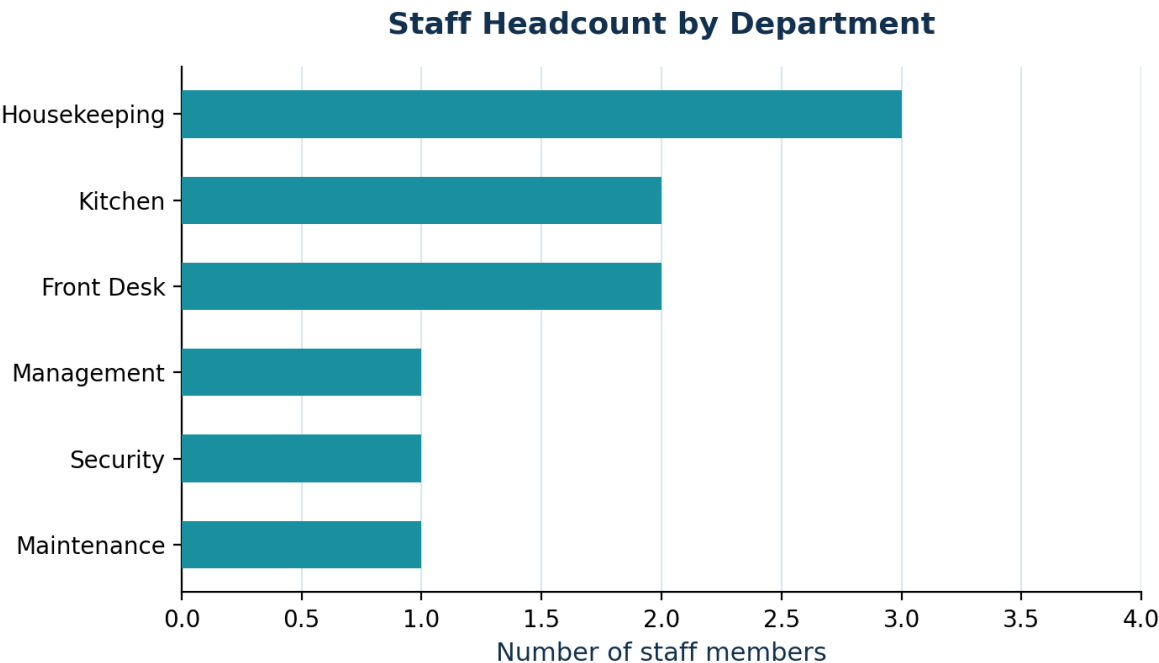


Figure 2. 10 seeded staff members across 6 departments, weighted toward Housekeeping, which carries the highest day-to-day headcount need in a small hotel.

4.3 Bookings dataset

60 live bookings are seeded from a Kaggle-style CSV shaped like the well-known “Hotel Booking Demand” dataset (lead time, ADR, market segment, country, cancellation status). Rows marked canceled in the source data are skipped, and each accepted row is mapped to a real room of the matching type with randomized, near-present check-in/out dates so the seeded bookings always look current relative to the day the app is installed.

4.4 Attendance history

30 days of attendance are generated per staff member (Sundays skipped): a 5% chance of an absence, otherwise a check-in at either 9:00 (present) or 10:00 (late) and 7.5–9 hours worked, giving payroll a realistic base to calculate from immediately.

5. Automation: The Autopay Engine

Autopay is the project's core piece of business logic. For a given month, each active staff member's attendance rows are pulled, days present and total hours are summed, and pay is derived directly from hours worked × hourly rate:

| Step       | Calculation                     | Notes                                           |
|------------|---------------------------------|-------------------------------------------------|
| Gross pay  | total_hours &times; hourly_rate | Hourly rate is pre-computed from monthly salary |
| Deductions | 2% of gross                     | Flat notional deduction, PF-like placeholder    |
| Net pay    | gross &minus; deductions        | What is disbursed                               |

| Step              | Calculation                                     | Notes                                           |
|-------------------|-------------------------------------------------|-------------------------------------------------|
| Disbursement      | Auto-marked "paid" with a transaction reference | Only if autopay_enabled=1 for that staff member |
| If autopay is off | Payroll row is created as "pending"             | Awaits manual admin approval (extension hook)   |

A daemon thread checks once an hour whether today is the 1st of the month and, if so and it hasn't already run this month, calculates and disburses payroll for every autopay-enabled staff member for the *previous* month — mirroring a real bank auto-debit mandate. An admin can also trigger the same calculation on demand from the Payroll screen with a "Run Autopay Now" button, which is useful for demos since it doesn't require waiting for the 1st of the month.

## 6. Key Insights

- Deriving payroll directly from attendance rather than a separate manual entry closes the gap between what actually happened on the floor and what staff are paid for — the two can no longer drift apart.
- Housekeeping is the largest single department in the seeded staff (3 of 10), reflecting real hotel staffing ratios where housekeeping typically outnumbers other departments.
- The per-staff autopay toggle is a useful middle ground: it lets the system fully automate disbursement for most staff while still leaving a manual-approval path open for exceptions.
- Because room type is assigned per floor rather than per room, occupancy and pricing patterns are easy to reason about for a demo, but this is a simplification a real property would not make.

## 7. Limitations

- The Flask secret key and demo passwords (*staff123*, *admin123*) are hard-coded and must be replaced before any real deployment.
- The 2% payroll deduction is a flat notional placeholder, not a real tax/PF calculation.
- Room assignment on booking simply grabs the first available room of the requested type; it does not yet check for date-range conflicts against existing bookings for that room.
- The background autopay thread's "has it run this month" state lives only in memory, so it resets if the server restarts on the 1st of the month.
- No automated test suite currently accompanies the route and payroll logic.

## 8. Recommendations & Next Steps

- Add a date-range availability check when assigning rooms to bookings to prevent double-booking.
- Replace the hard-coded secret key and demo credentials with environment-variable configuration before any deployment beyond a local demo.
- Persist the "last autopay run" marker in the database instead of an in-memory variable so it survives server restarts.
- Integrate a real payment gateway (e.g. Razorpay/Stripe) in place of the simulated transaction reference.
- Add role-based permissions so payroll and salary data are restricted to management-level admin accounts.
- Introduce automated tests around booking, check-in, attendance, and payroll calculation logic.



## Appendix: Project Metadata

| Item                | Detail                                                                                          |
|---------------------|-------------------------------------------------------------------------------------------------|
| Entry point         | app.py (Flask routes for guest, staff, admin)                                                   |
| Data layer          | database.py (schema definition + demo data seeding)                                             |
| Dependencies        | Flask >= 3.0.0, Werkzeug >= 3.0.0 (see requirements.txt)                                        |
| Database file       | instance/happystayz.db (auto-created; delete to reset & reseed)                                 |
| Demo dataset source | data/hotel_bookings.csv, generated in the same schema as Kaggle's "Hotel Booking Demand" (Anton |
| Run command         | python3 app.py → http://127.0.0.1:5000                                                          |